🚀 AWS EBS CSI Driver Installation in EKS — Complete Step by Step Explanation

🔹 Step 1: Associate OIDC Provider

- 🔸 By default, Kubernetes Pods cannot access AWS APIs. We need a secure way for Pods to assume AWS IAM Roles.
- 🔸 AWS uses OIDC Provider for this.

```
eksctl utils associate-iam-oidc-provider\
--cluster eks-cluster2\                          #EKS Cluster Name
--region ap-south-1 \
--approve
```

- ✅ Verify :

```
aws eks describe-cluster \
--name eks-cluster2 \
--region ap-south-1 \
--query "cluster.identity.oidc.issuer"
```

🔹 Step 2: IAM Service Account (IRSA)

- 🔸 The EBS CSI Driver must call AWS APIs for `CreateVolume`, `DeleteVolume`, `AttachVolume` and `DetachVolume`.
- ❌ Without permissions: `PVC = Pending`
- 🔸 AWS provides a managed policy : `AmazonEBSCSIDriverPolicy` This policy already contains all required permissions for the `EBS CSI Driver.`
- 🔸 Create IAM Service Account (IRSA) `IAMRole + Kubernetes Service Account`

```
IAM Role
      ↓
Kubernetes Service Account
      ↓
EBS CSI Driver Pod
```

- 💻 Command

```
eksctl create iamserviceaccount \
--name ebs-csi-controller-sa \
--namespace kube-system \
--cluster eks-cluster2 \
--region ap-south-1 \
--role-name AmazonEKS_EBS_CSI_DriverRole \
--attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
--approve
```

- 📋 What each option does :

🛠️ Option	🎯 Purpose
`eksctl create iamserviceaccount`	Creates a Kubernetes ServiceAccount and links it to an AWS IAM Role (IRSA).
`--name ebs-csi-controller-sa`	ServiceAccount name used by the EBS CSI Driver controller pod.
`--namespace kube-system`	Creates the ServiceAccount in the kube-system namespace.
`--cluster eks-cluster2`	Target EKS cluster name.
`--region ap-south-1`	AWS region where the cluster exists.
`--role-name AmazonEKS_EBS_CSI_DriverRole`	IAM Role name to create.
`--attach-policy-arn AmazonEBSCSIDriverPolicy`	Attaches AWS-managed policy containing EBS permissions.
`--approve`	Immediately creates the resources. Without this, it only shows what will be created.

🔷 Step 3 : Install EBS CSI Driver Addon

- 🔸 This installs the `CSI Driver Pods.`
- ❌ Without CSI Driver: `PVC = No Volume Creation`
- 💻 Command Add (`AWS Account ID)`

```
aws eks create-addon \
--cluster-name eks-cluster2 \
--addon-name aws-ebs-csi-driver \
--service-account-role-arn arn:aws:iam::<ACCOUNT-ID>:role/AmazonEKS_EBS_CSI_DriverRole
--resolve-conflicts OVERWRITE
```

- ✅ Verify

```
aws eks describe-addon \
--cluster-name eks-cluster2 \
--addon-name aws-ebs-csi-driver \
--query addon.status
```

🔹 Step 4: Verify CSI Driver

🛠️ Command	✅ Expected Output	🎯 Purpose
`kubectl get csidrivers`	`ebs.csi.aws.com`	Confirms that the Amazon EBS CSI Driver is installed and registered with the Kubernetes cluster.
`kubectl getpods -n kube-system| grep ebs`	`ebs-csi-controller`	Confirms that the CSI Controller component is running and can communicate with AWS APIs to create, attach, and delete EBS volumes.
`kubectl getpods -n kube-system| grep ebs`	`ebs-csi-node`	Confirms that the CSI Node component is running on worker nodes and can mount EBS volumes to Pods.

🔹 Step 5: Create StorageClass

- 🔸 StorageClass tells Kubernetes: `Which storage?,` `Which driver?` and `What type?`
- 📄 `storageclass.yaml`

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

- ▶ apply : kubectl apply -f storageclass.yaml

🔹 Step 6: Create PVC

- 🔸 PVC = Storage Request
- 📄 pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: ebs-gp3
  resources:
    requests:
      storage: 5Gi
```

- ▶ apply : kubectl apply -f pvc.yaml
- ⏳ Initially: `STATUS: Pending`
- 🔸 Normal because: `volumeBindingMode: WaitForFirstConsumer`

🔹 Step 7: Create Pod

```
apiVersion:    v1
kind:          Pod
metadata:
  name: nginx-ebs
spec:
  containers:
  - name: nginx
    image: nginx

    volumeMounts:
    - mountPath: /data
      name: ebs-storage

  volumes:
  - name: ebs-storage
      persistentVolumeClaim:
        claimName: mysql-pvc
```

- ▶ Apply : `kubectl apply -f pod.yaml`

🔹 Step 8: Dynamic Provisioning Happens

📌 When Pod starts:

```
Pod Created
↓
PVC Requested
↓
StorageClass Selected
↓
EBS CSI Driver Called
↓
AWS CreateVolume API
↓
New EBS Volume Created
↓
PV Created Automatically
↓
PVC Bound
↓
Pod Mounted Storage
```

🔹 Step 9: Verify

```
kubectl get pvc
kubectl get pv
aws ec2 describe-volumes --region ap-south-1
```

🎤 Interview Answer (1 Minute)

- 🔹 In EKS, I first associated the `OIDC provider` and created an `IAM Service Account` with the `AmazonEBSCSIDriverPolicy`.
- 🔹 Then I installed the `AWS EBS CSI Driver addon`. After that, I created a StorageClass using `ebs.csi.aws.com`, created a `PVC`, and deployed a Pod.
- 🔹 The CSI Driver dynamically created an AWS EBS volume, generated a PV automatically, bound it to the PVC, and mounted the storage into the Pod.
- 🔹 This is called `dynamic volume provisioning` in Kubernetes.